Week 3 SQL Advanced Analytics – Query Results

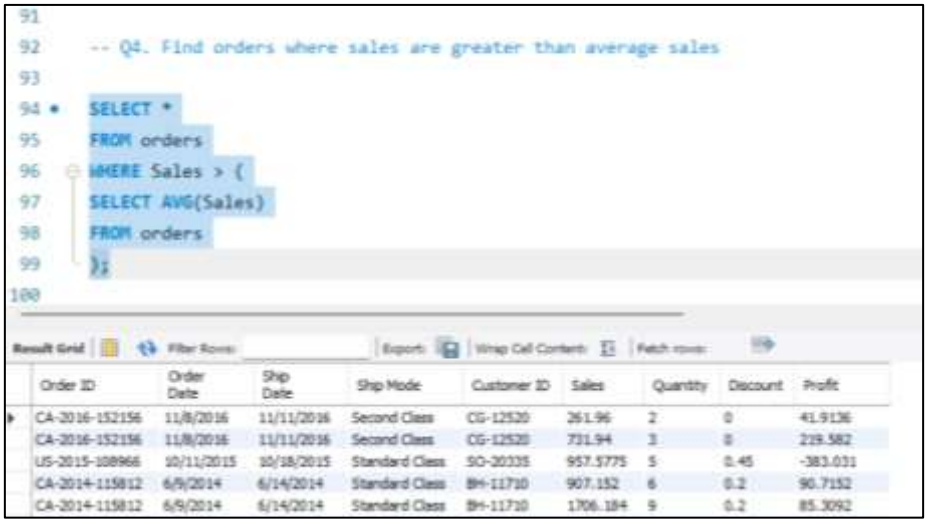
(These queries are included in the advanced_sql_queries.sql file.
The following queries are from Q4 to Q9 and Q11.)

1. Find all orders where sales are greater than the average sales

SQL Query:

```
SELECT *
FROM orders
WHERE Sales > (
SELECT AVG(Sales)
FROM orders
);
```

Output:

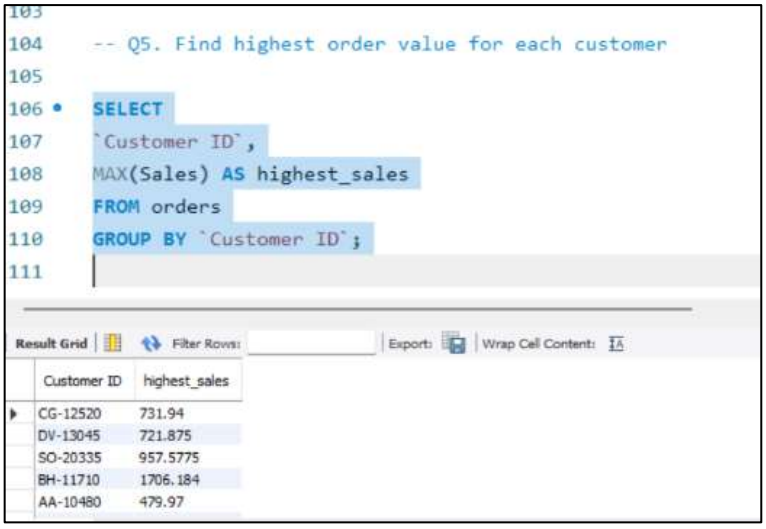


2. Find the highest sales order for each customer

SQL Query:

```
SELECT
`Customer ID`,
MAX(Sales) AS highest_sales
FROM orders
GROUP BY `Customer ID`;
```

Output:

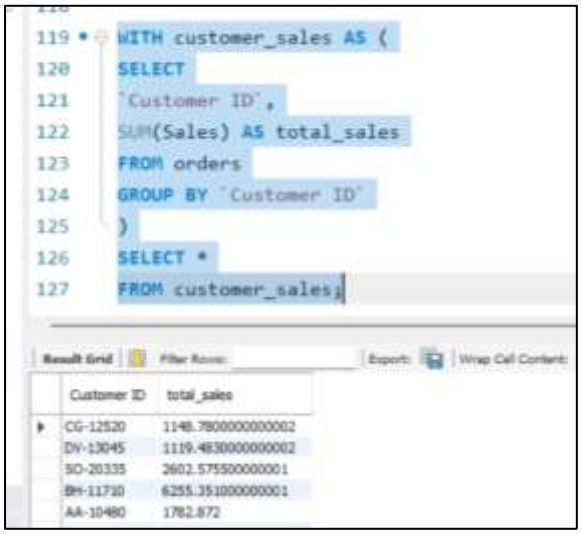


3. Calculate total sales for each customer using CTE

SQL Query:

```
WITH customer_sales AS (
SELECT
`Customer ID`,
SUM(Sales) AS total_sales
FROM orders
GROUP BY `Customer ID`
)SELECT *
FROM customer_sales;
```

Output:

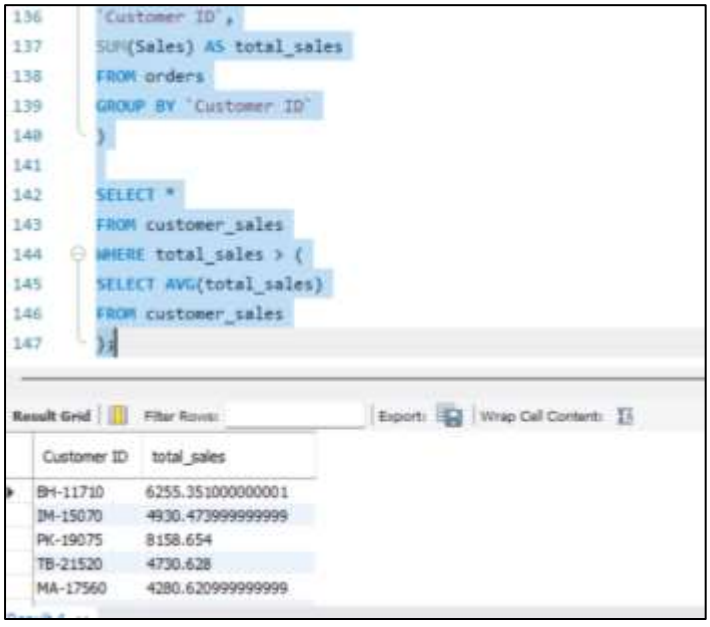


4. Find customers whose total sales are above average

SQL Query:

```
WITH customer_sales AS (
SELECT
`Customer ID`,
SUM(Sales) AS total_sales
FROM orders
GROUP BY `Customer ID`
)
SELECT *
FROM customer_sales
WHERE total_sales > (
SELECT AVG(total_sales)
FROM customer_sales
);
```

Output:



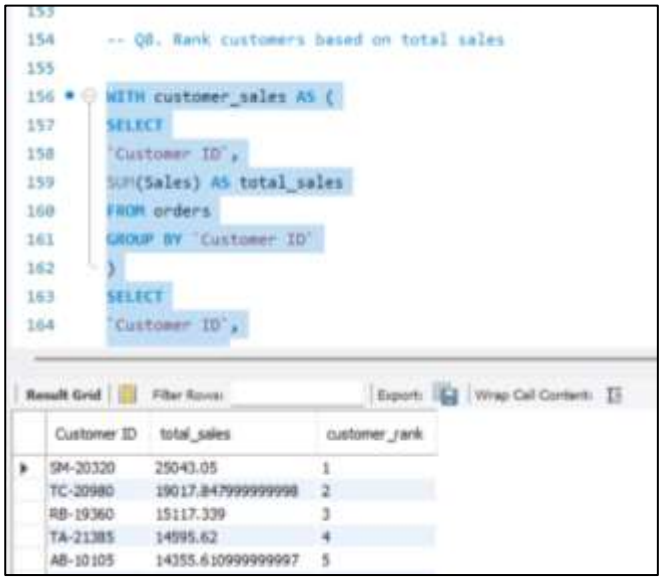
5. Rank all customers based on total sales

SQL Query:

```
WITH customer_sales AS (
```

```
SELECT
`Customer ID`,
SUM(Sales) AS total_sales
FROM orders
GROUP BY `Customer ID`
)
SELECT
`Customer ID`,
total_sales,
RANK() OVER (ORDER BY total_sales DESC) AS customer_rank
FROM customer_sales;
```

Output:



6. Assign row numbers to each order within a customer

SQL Query:

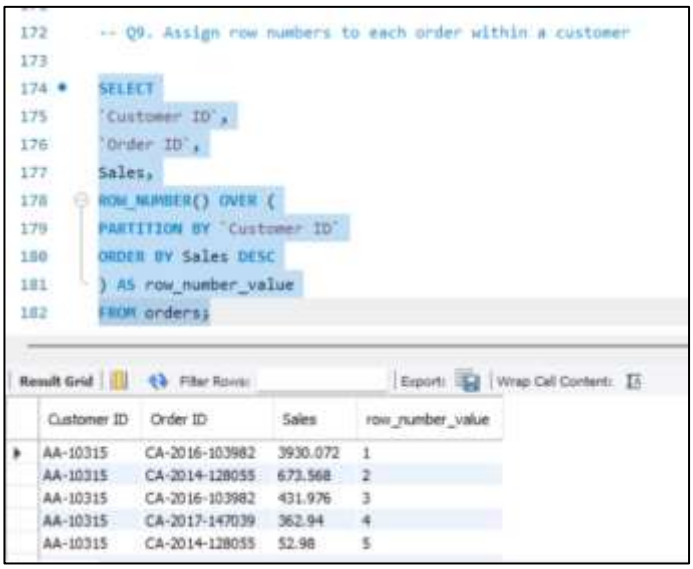
```
SELECT
`Customer ID`,
`Order ID`,
Sales,
ROW_NUMBER() OVER (
PARTITION BY `Customer ID`
```

ORDER BY Sales DESC

) AS row_number_value

FROM orders;

Output:



7. Display top 3 customers based on total sales

SQL Query:

WITH customer_sales AS (

SELECT

`Customer ID`,

SUM(Sales) AS total_sales

FROM orders

GROUP BY `Customer ID`

)

SELECT

`Customer ID`,

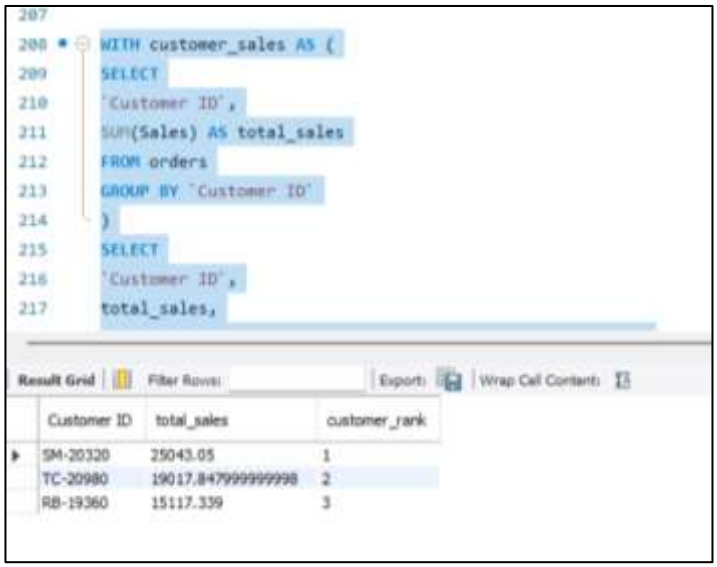
total_sales,

RANK() OVER (ORDER BY total_sales DESC) AS customer_rank

FROM customer_sales

LIMIT 3;

Output:



Final Combined Query

(Using JOIN + CTE + Window Function)

SQL Query:

```
WITH customer_sales AS (
SELECT
`Customer ID`,
SUM(Sales) AS total_sales
FROM orders
GROUP BY `Customer ID`
)
SELECT
c.`Customer Name`,
cs.total_sales,
RANK() OVER (ORDER BY cs.total_sales DESC) AS customer_rank
FROM customer_sales cs
```

JOIN customers c

ON cs.`Customer ID` = c.`Customer ID`;

Output: